



Module 15: Scheduling Algorithms

CPE010 DATA STRUCTURES AND ALGORITHMS

Learning Preemptive and Non Preemptive Scheduling Algorithms

Intended Learning Outcomes

After this module, the student should be able to:

- Create C++ code to implement non-preemptive and preemptive scheduling algorithms.
- Analyze use-cases and compare different scheduling algorithms.

Introduction

- CPU Scheduling algorithms are used to assign system resources to processes in a computing system.

Sample Situation:

Consider the case where you are using two apps namely a game like *Fortnite* and a desktop application like *Evernote*.

Both with require the use of a graphics processor and but only one can use it at a time.

- The CPU scheduling algorithms aims to:
 - Manages which process will use a given resource at a time.
 - Maximize CPU resource usage,
 - Minimize waiting time for each process.

CPU Scheduling

The **CPU Scheduler** selects from among the processes in memory that are ready to execute, and allocates the CPU to one of them

CPU scheduling decisions may take place when a process:

1. Switches from running to waiting state
2. Switches from running to ready state
3. Switches from waiting to ready
4. Terminates

Scheduling under 1 and 4 is **non-preemptive**. All other scheduling is **preemptive**.

Scheduling Criteria

- **CPU utilization** – keep the CPU as busy as possible
- **Throughput** – # of processes that complete their execution per time unit
- **Turnaround time** – amount of time to execute a particular process
- **Waiting time** – amount of time a process has been waiting in the ready queue
- **Response time** – amount of time it takes from when a request was submitted until the first response is produced, not output (for time-sharing environment)
- Optimization Criteria:

Max CPU Utilization	Max throughput
Max turnaround time	Min waiting time
Min response time	

Scheduling Techniques

- **Preemptive Scheduling Algorithms** are those which are based on the priority of the processes.
 - By preference, when a high priority process enters, it preempts a low priority process in between and executes the high priority process first.
- **Non-preemptive Scheduling Algorithms** are those who can't be preempted in between.
 - i.e. we can not take control of CPU in between until the current process completes its execution.

First-Come, First-Served (FCFS) Scheduling

Process	Burst Time
P_1	24
P_2	3
P_3	3

Suppose that the processes arrive in the order: P_1 , P_2 , P_3 . The Gantt Chart for the schedule is:



Waiting time for $P_1 = 0$; $P_2 = 24$; $P_3 = 27$

Average waiting time: $(0 + 24 + 27)/3 = 17$

First-Come, First-Served (FCFS) Scheduling

Process	Burst Time
P_1	24
P_2	3
P_3	3

Suppose that the processes arrive in the order P_2, P_3, P_1 . The Gantt chart is:



Determine the **waiting time** for: $P_1 = \underline{\hspace{2cm}}$, $P_2 = \underline{\hspace{2cm}}$, $P_3 = \underline{\hspace{2cm}}$

Average waiting time is $\underline{\hspace{2cm}}$.

First-Come, First-Served (FCFS) Scheduling

Process	Burst Time
P ₁	24
P ₂	3
P ₃	3

Suppose that the processes arrive in the order P₂, P₃, P₁. The Gantt chart is:



Waiting time for **P1 = 6; P2 = 0; P3 = 3**

Average waiting time: **$(6 + 0 + 3)/3 = 3$**

Much better than previous case.

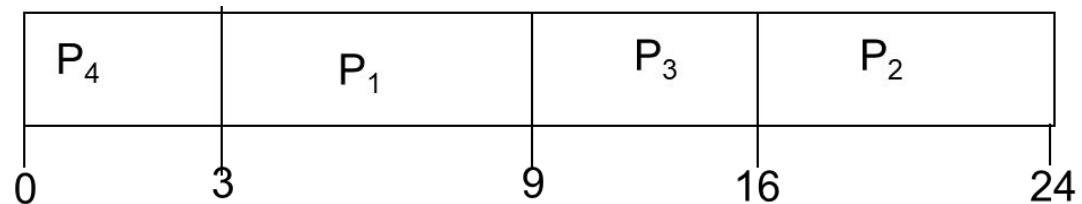
Shortest-Job-First (SJF) Algorithm

- Associate with each process the length of its next CPU burst. Use these lengths to schedule the process with the shortest time.
- **SJF is optimal** – gives minimum average waiting time for a given set of processes
- The **difficulty** is knowing *the length of the next CPU request*.

How do we determine this?

Process	Burst Time
P1	6
P2	8
P3	7
P4	3

The SJF Scheduling Chart would be...



What is the **average waiting time**?

Shortest Remaining Time First (SRTF)

- The Preemptive version of Shortest Job First(SJF) scheduling.
- The processes are scheduled according to the shortest remaining time.
- In the SRTF scheduling algorithm, the execution of any process can be stopped after a certain amount of time.

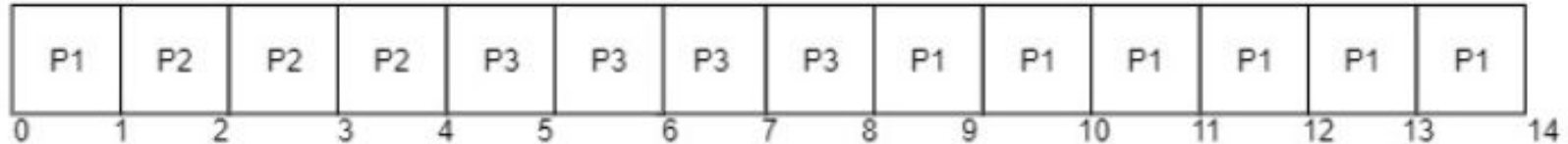
Process	Burst Time	Arrival Time
P1	7	0
P2	3	1
P3	4	3

The Gantt Chart would be...

Shortest Remaining Time First (SRTF)

Process	Burst Time	Arrival Time
P1	7	0
P2	3	1
P3	4	3

The Gantt Chart would be...



As Arrival Time and Burst time for three processes P1, P2, P3 are given in the above diagram. Let us calculate Turnaround time, completion time, and waiting time.

SRTF Calculations

Process	Arrival Time	Burst Time	Completion Time	Turnaround Time
P1	0	7	14	$14 - 0 = 14$
P2	1	3	4	$4 - 1 = 3$
P3	3	4	8	$8 - 3 = 5$

average waiting time = waiting for time of all processes / no.of processes

average waiting time = $7 + 0 + 1 = 8 / 3 = \mathbf{2.66 \text{ ms}}$

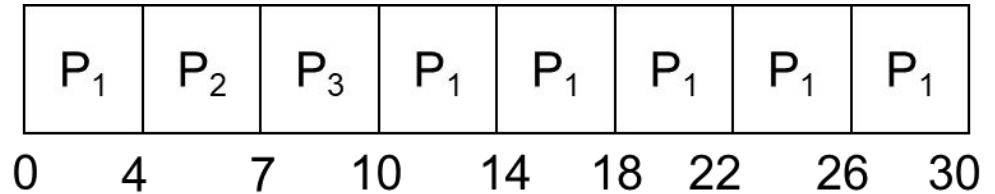
Round Robin (RR)

- Each process gets a small unit of CPU time (**time quantum**), usually 10-100 milliseconds. After this time has elapsed, the process is preempted and added to the end of the ready queue.
- We can predict wait time:
 - If there are n processes in the ready queue and the time quantum is q , then each process gets $1/n$ of the CPU time in chunks of at most q time units at once.
 - No process waits more than $(n-1)q$ time units.
- Performance
 - q large $\rightarrow$ FIFO
 - q small $\rightarrow$ may hit the context switch wall: q must be large with respect to context switch, otherwise overhead is too high

Round Robin (RR)

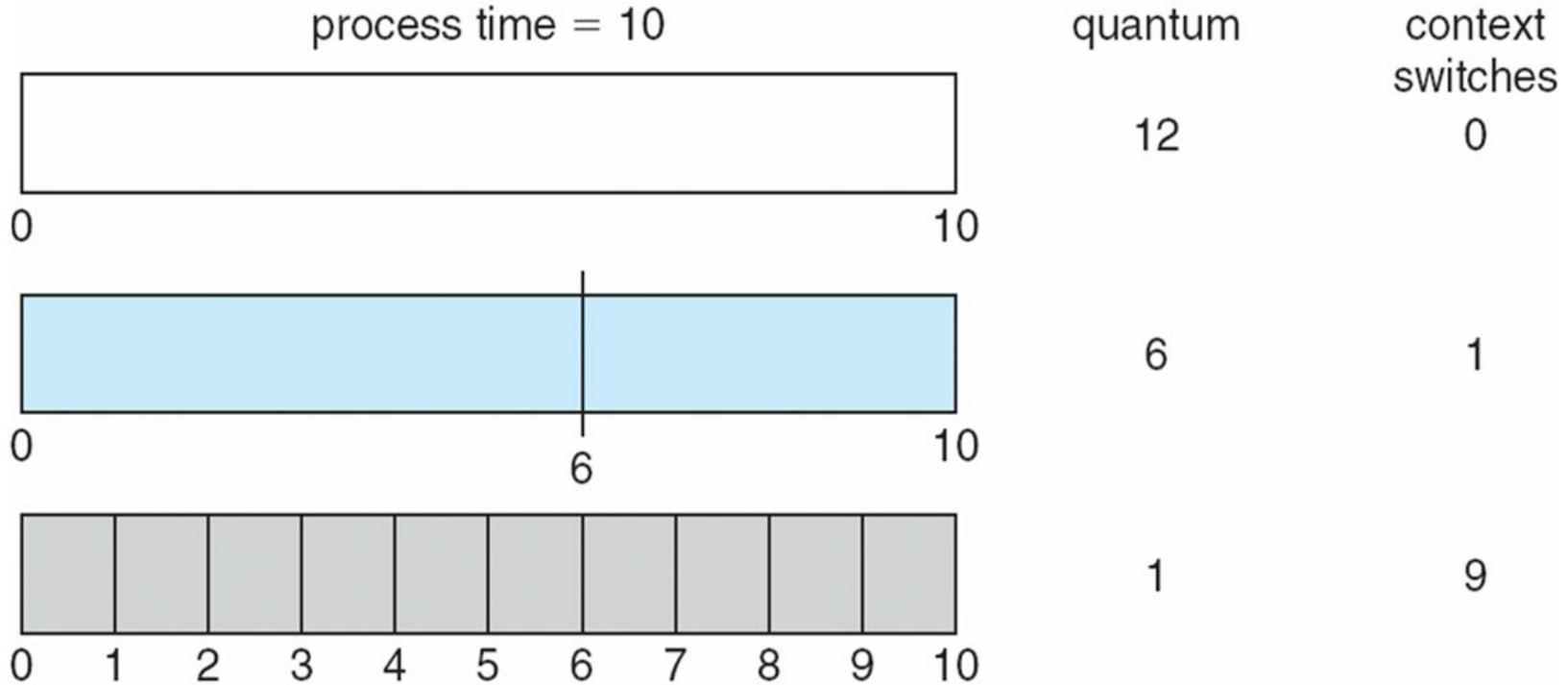
Process	Burst Time
P1	24
P2	3
P3	3

The Gantt Chart would be...



Typically, higher average turnaround than SJF, but better response.

Time Quantum and Process Switching





Thanks for participating!

Submit your questions through the discussion
Q&A found on module 15.